



UNIVERSITÉ
LIBRE
DE BRUXELLES

08 Mai 2026

Software Requirement Document

Étudiants:

ADIYA, C.

BENZIANI, N.

BEATRIZ DE ASSUNÇÃO, T.

CONTULIANO BRAVO S.

KANOUN, A.

Assistant académique:

HUGO CALLEBAUT

Table des matières

Liste des fig.s	II
1 Introduction	1
2 Architecture du système	2
2.1 La vue globale du système	2
2.2 L'architecture client-serveur	2
3 Fonctionnalités du système	4
3.1 L'authentification	4
3.2 La gestion des projets	5
3.3 Exportation au format standard (PNG, BMP,...) ou format natif	6
3.4 Importation d'un projet au format interne	7
3.5 La création, partage et la suppression de projets	8
3.6 Les permissions et les rôles à l'intérieur d'un projet	9
3.7 Le chat	10
4 Edition de la carte	11
4.1 La navigation et la carte	11
4.2 Les outils et les couches	13
4.3 Le remplissage automatique	15
5 Spécification techniques	16
5.1 Stockage des données	16
5.2 Gestion de la concurrence des modifications	17
5.3 Gestion du flux de données	17
6 Annexes	18
Bibliographie	19

Liste des fig.s

Fig. 2.1 Diagramme de classe du serveur	3
Fig. 3.1 Diagramme d'activité pour la connexion	4
Fig. 3.2 Use case des autorisations d'un utilisateur	5
Fig. 3.3 Exportation d'un projet au format natif (PNG, SVG, ...)	6
Fig. 3.4 Exportation d'un projet au format interne	6
Fig. 3.5 Importation d'un projet au format interne	7
Fig. 3.6 Diagramme de séquence des actions possible sur un projet	8
Fig. 3.7 Use case illustrant les actions possible dans un projet	9
Fig. 3.8 Diagramme de class pour les messages	10
Fig. 4.1 Diagramme de séquence du déplacement et du zoom	11
Fig. 4.2 Diagramme de class de l'affichage des couches, déplacement et zoom	12
Fig. 4.3 Use Case des outils	13
Fig. 4.4 Diagramme de classe des outils et couches	14
Fig. 4.5 Diagramme de séquence de l'utilisation des outils	15
Fig. 4.6 Squence Diagram Automatic Filling	15
Fig. 5.1 Diagramme de séquence de la sauvegarde automatique	16
Fig. 5.2 Diagramme de séquence sur les modification d'un projet	17
Fig. 5.3 Diagramme de séquence sur les modification d'un projet	17

1 Introduction

Ce document est destiné au développement d'une application d'éditeur de cartes RPG, qui est collaborative. L'application permet à plusieurs utilisateurs de travailler ensemble de manière simultanée, en garantissant la cohérence des données et la synchronisation entre ceux-ci.

L'application repose sur une architecture client-serveur. D'une part, il y a le client, qui gère l'interface utilisateur et l'utilisation des fonctionnalités. D'autre part, il y a le système qui gère la centralisation des données et l'envoi des modifications aux autres utilisateurs qui sont connectés au projet. Ce document présente l'architecture globale du système et les principales fonctionnalités accessibles à l'utilisateur.

2 Architecture du système

2.1 La vue globale du système

Le système se base sur une architecture client-serveur, chaque client interagit avec un seul serveur central. Le client va permettre à l'utilisateur de visualiser et modifier une carte via une interface graphique, alors que le serveur gère l'interaction entre les utilisateurs et le stockage des données.

D'abord, les actions réalisées par un utilisateur sont envoyées au serveur via un socket de la bibliothèque "SFML". Ensuite, le serveur va traiter les modifications de l'utilisateur, mettre à jour ces données si nécessaire, et notifie les autres utilisateurs connectés des modifications effectuées pour une synchronisation entre ceux-ci.

2.2 L'architecture client-serveur

Le client transmet ses commandes au serveur sous forme de requêtes. Le serveur centralise ces actions : il met à jour la base de données et synchronise l'état du projet en informant les autres utilisateurs connectés. Ces échanges sont entièrement asynchrones, ce qui permet de ne jamais bloquer l'application en attendant une réponse du réseau.

La communication réseau est entièrement encapsulé par deux classes managers. Cette architecture permet de masquer la complexité technique au reste de l'application.

Les deux classes responsables sont :

1. `ServeurNetworkManager` : Gère le socket d'écoute et maintient la liste des clients connectés. Il centralise la réception des messages et les distribue dans une file d'événements serveur.
2. `ClientNetworkManager` : Convertit les actions du jeu en requêtes réseau et,

inversement, réceptionne les messages du serveur pour les stocker dans une file d'événements locale. Cette file est ensuite traitée par la boucle principale du jeu pour mettre à jour les modifications



Remarque

Le nombre important de méthodes dans ces classes est volontaire. Il permet d'exposer chaque fonctionnalité réseau sous forme d'une commande simple, facilitant l'intégration avec le reste du code.

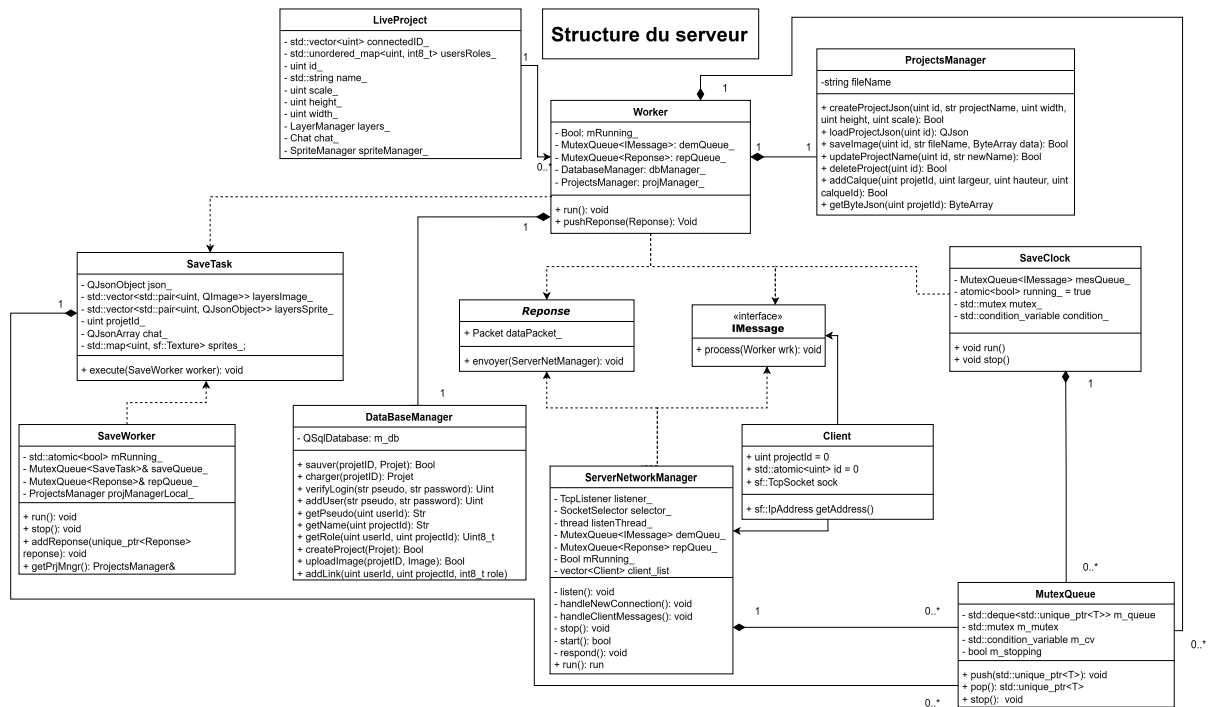


Fig. 2.1. – Diagramme de classe du serveur

3 Fonctionnalités du système

3.1 L'authentification

En rentrant dans le projet, l'utilisateur devra passer par une phase d'authentification, c'est ce qu'il lui permettra d'accéder au menu principal qui contient la liste des projets accessibles. Il aura le choix entre se connecter ou s'identifier, s'il choisit la connexion, il devra rentrer un identifiant et un mot de passe valide qui seront vérifiés par le serveur. S'il choisit la création d'un compte, il devra choisir un identifiant non utilisé (entre 3 et 15 caractères) et un mot de passe.

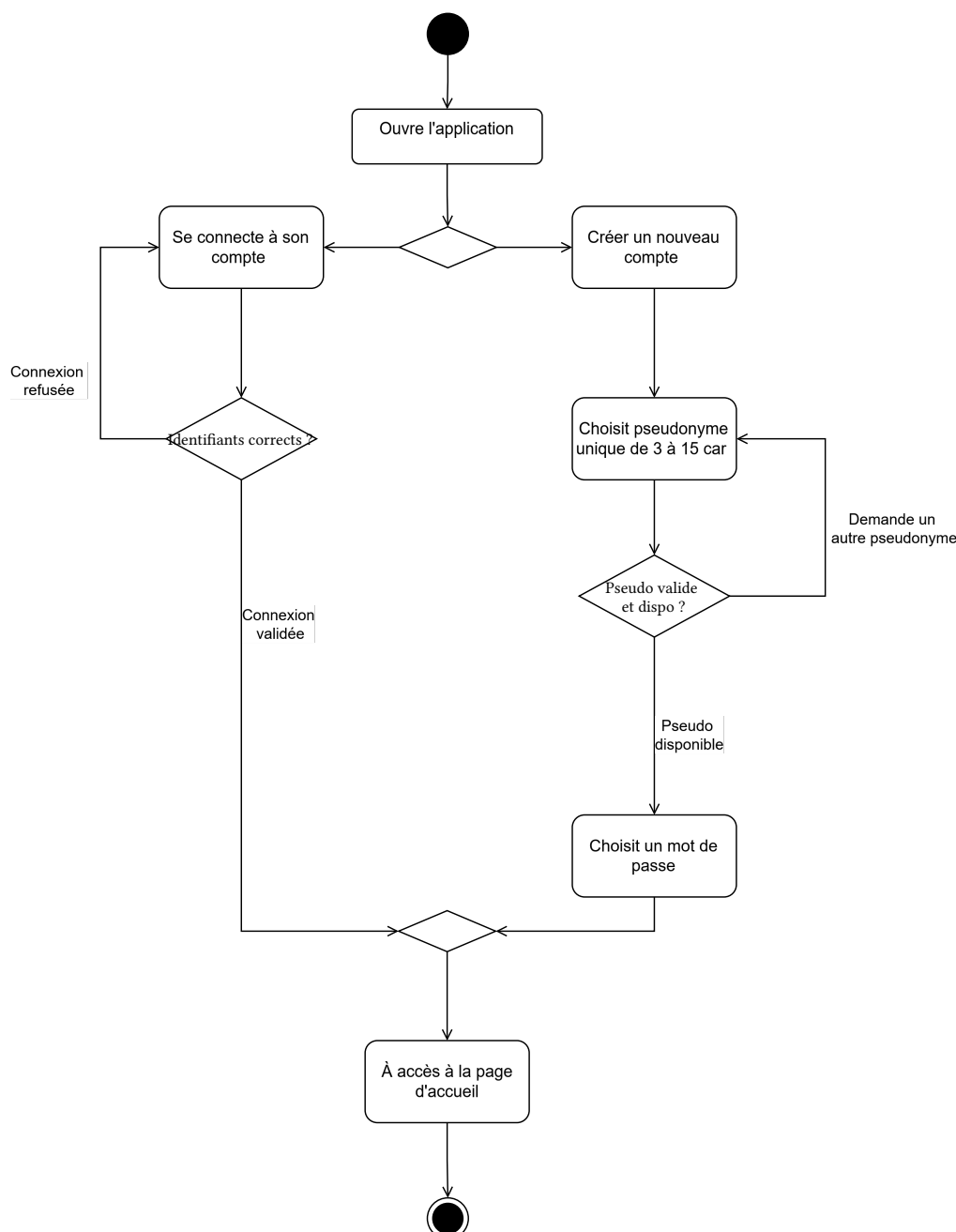


Fig. 3.1. – Diagramme d'activité pour la connexion

3.2 La gestion des projets

Use case relatif à toutes les actions qu'un utilisateur peut faire sur les projets auxquels il participe en fonction de son rôle dans chaque projet.

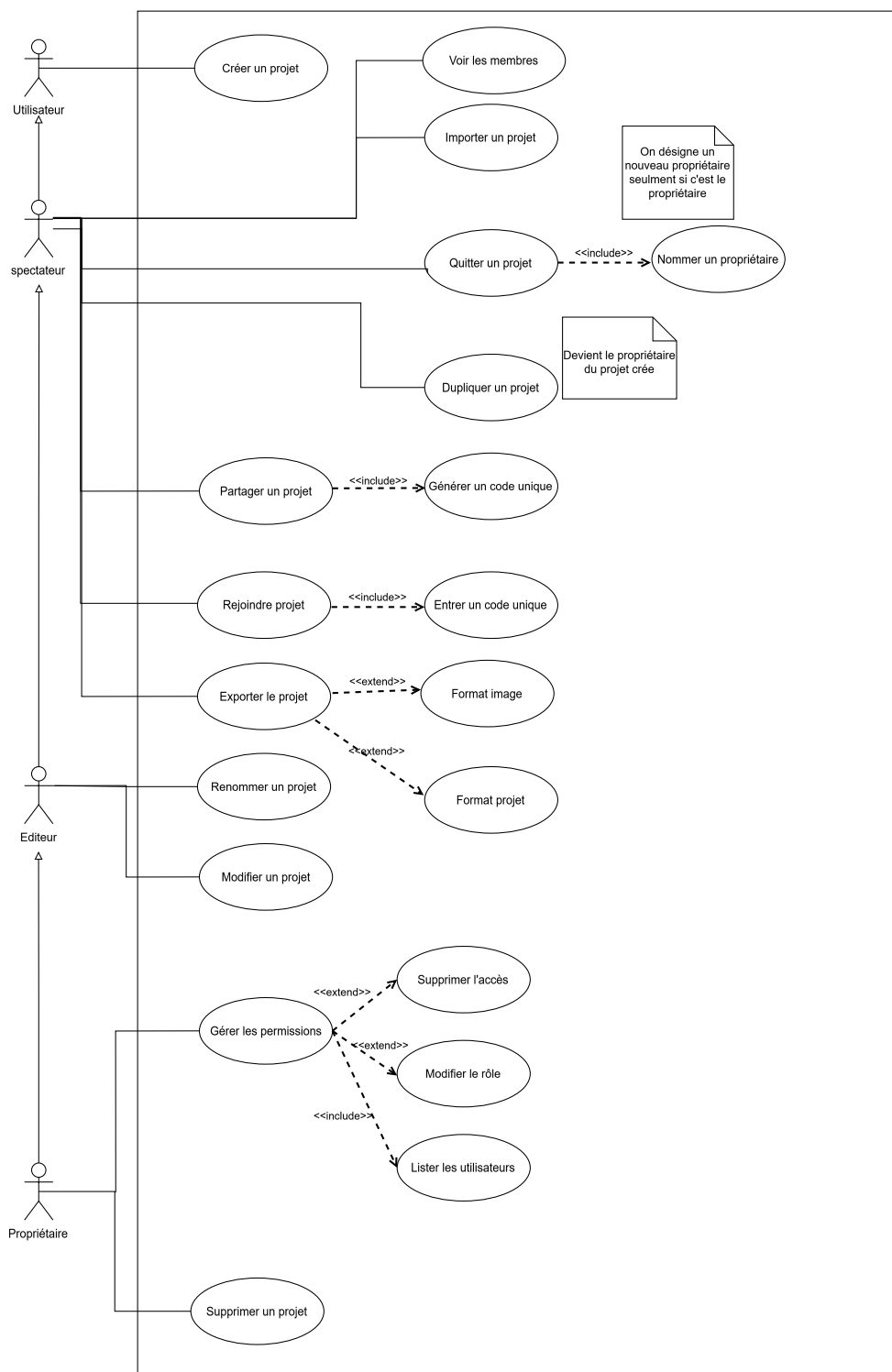


Fig. 3.2. – Use case des autorisations d'un utilisateur

3.3 Exportation au format standard (PNG, BMP,...) ou format natif

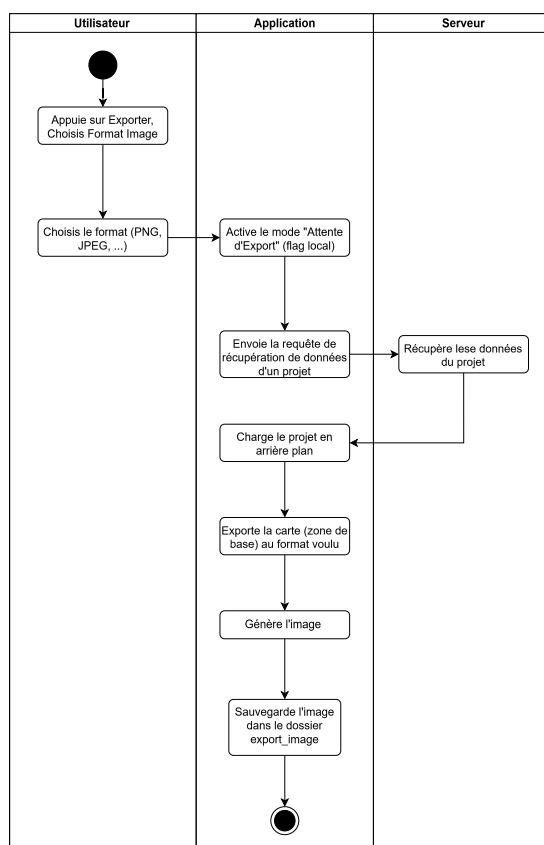


Fig. 3.3. – Exportation d'un projet au format natif (PNG, SVG, ...)

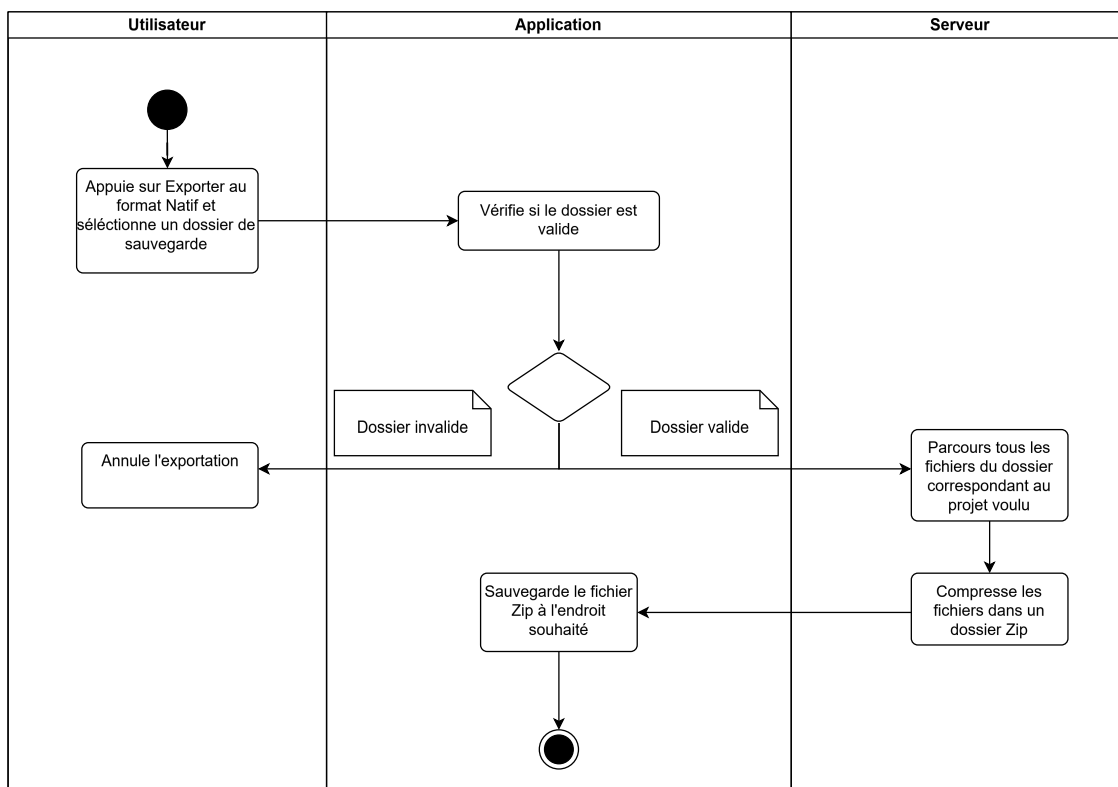


Fig. 3.4. – Exportation d'un projet au format interne

3.4 Importation d'un projet au format interne

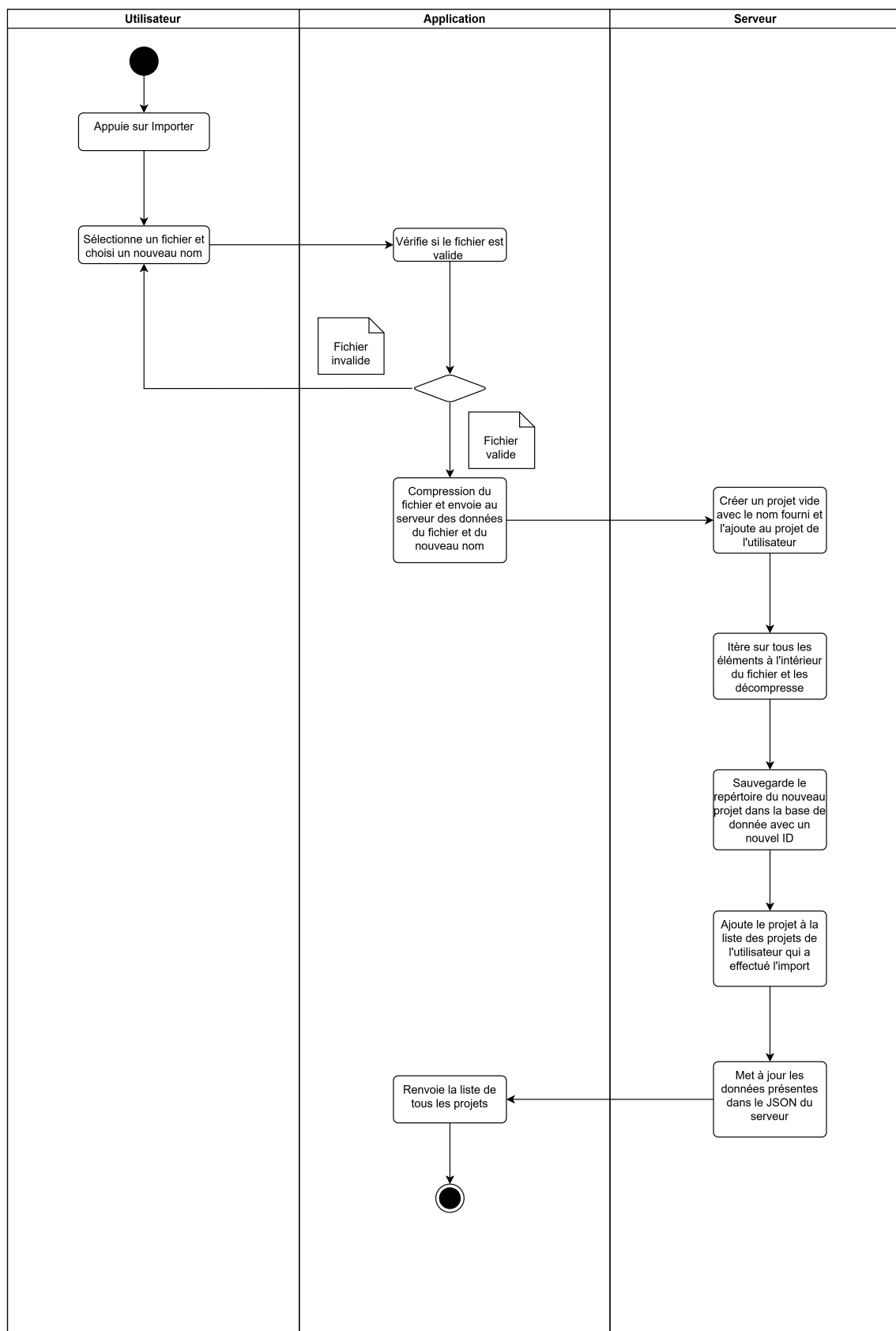


Fig. 3.5. – Importation d'un projet au format interne

3.5 La création, partage et la suppression de projets

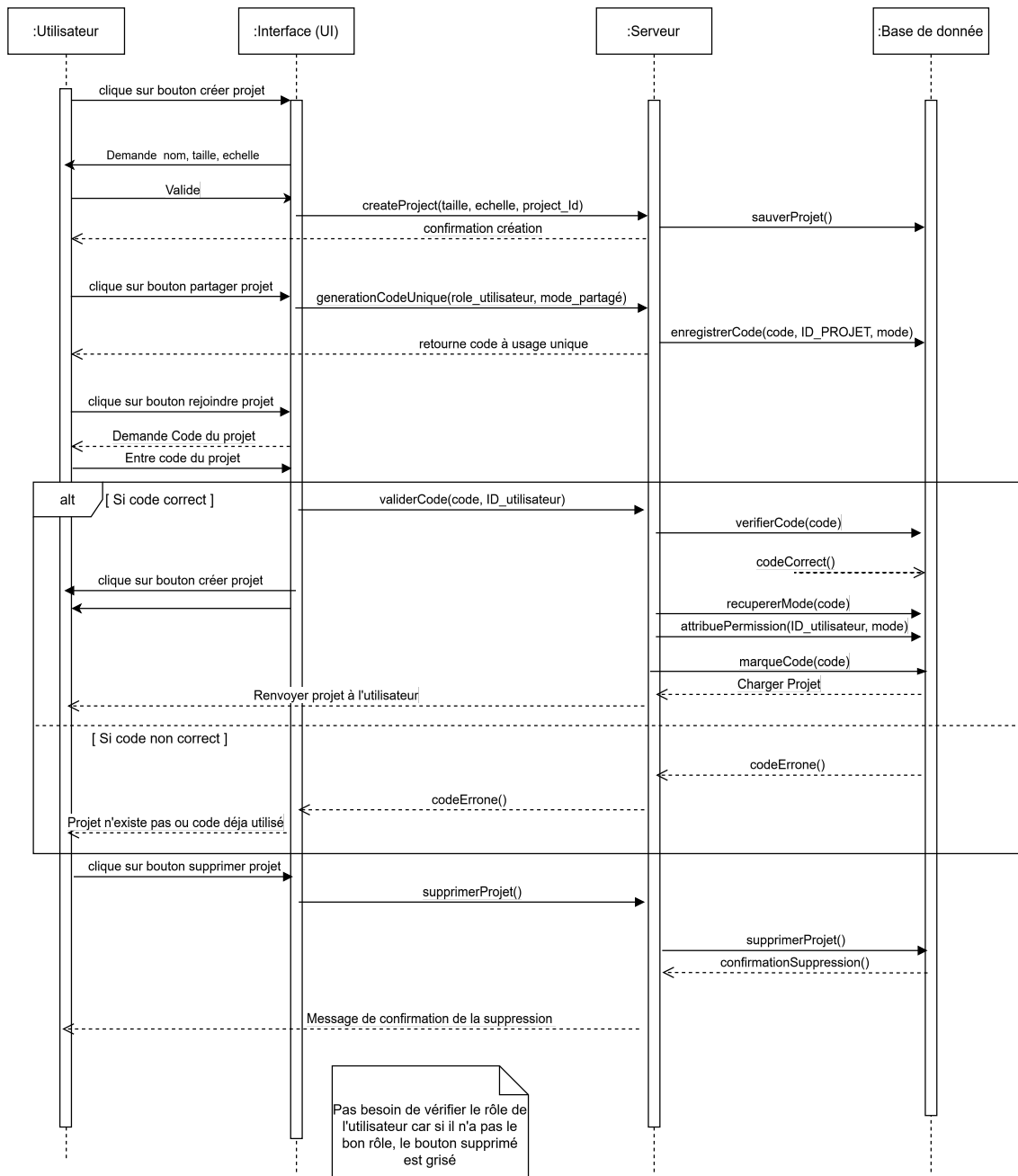


Fig. 3.6. – Diagramme de séquence des actions possible sur un projet

3.6 Les permissions et les rôles à l'intérieur d'un projet

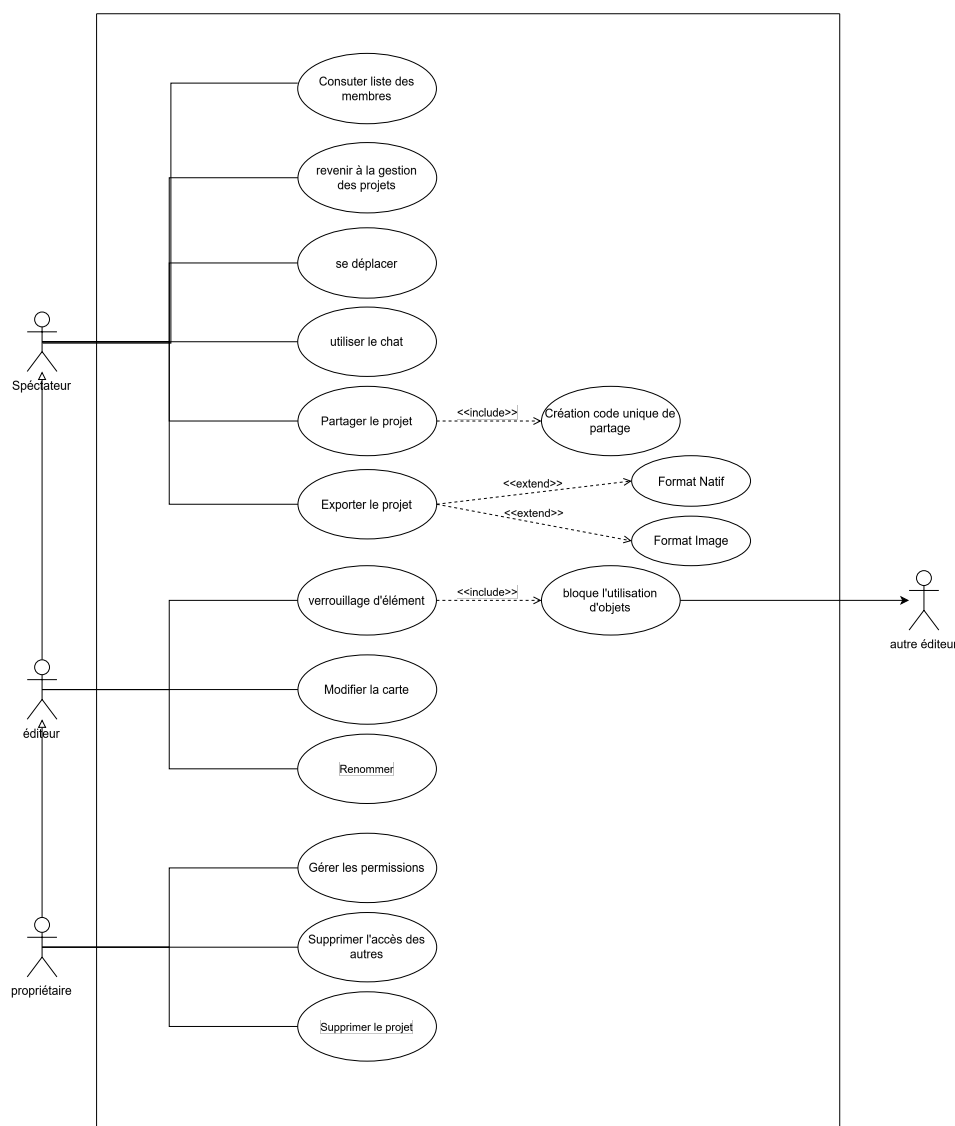


Fig. 3.7. – Use case illustrant les actions possible dans un projet

Dans un projet, chaque utilisateur a un rôle spécifique, qui détermine les actions qu'il lui est possible de faire. Propriétaire est le rôle qui contient le plus de droits. Les rôles sont les suivants:

- Propriétaire: il possède tous les droits dans un projet, il peut modifier la carte, supprimer le projet et gérer les rôles des utilisateurs participant au projet.
- Éditeur: peut modifier le contenu de la carte.
- Spectateur: est en lecture seule de la carte.

Tous les rôles ont la possibilité d'utiliser le chat. De plus, les permissions sont gérées par le client, mais aussi par le serveur afin d'éviter des actions non autorisées.

3.7 Le chat

Dans un projet, tous les utilisateurs qui ont accès au projet disposent d'un chat commun qui leur permet de discuter entre eux. C'est par celui-ci que les notifications système sont aussi affichées, telles que la connexion ou la déconnexion d'un utilisateur dans un projet, le verrouillage, le masquage, etc.

Le chat est représenté par la classe `Chat` qui stocke l'ensemble des messages qui ont été échangés dans le projet. Chaque message est de type `MessageChat`, chaque message contient le contenu du message, l'auteur et la date (seulement pour les messages utilisateurs). Il existe 2 types de messages : les `UserMessage` pour les messages rédigés par un utilisateur, et `SystemNotification` pour les événements générés automatiquement par le serveur.

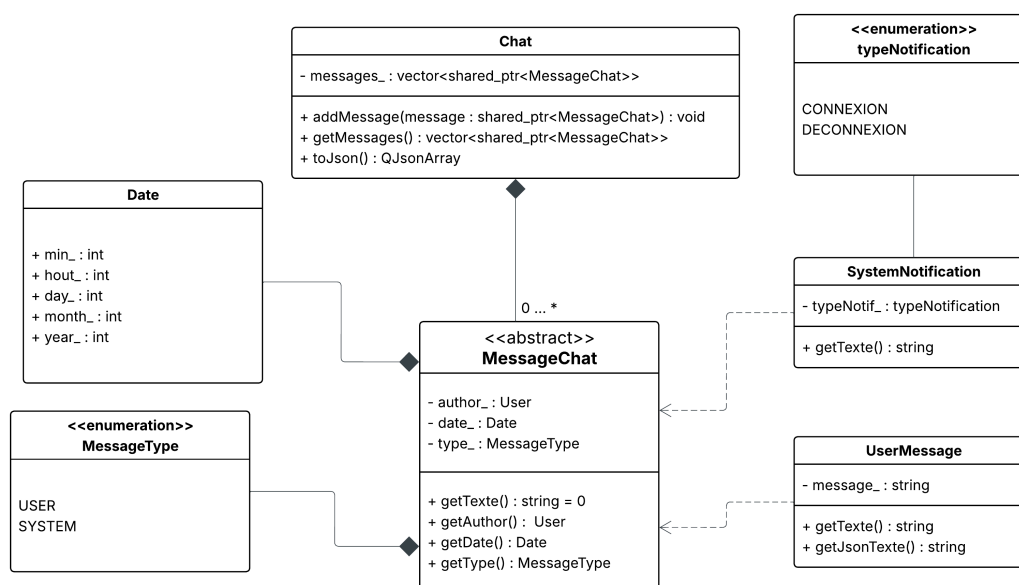


Fig. 3.8. – Diagramme de class pour les messages

Quand un utilisateur veut envoyer un message dans le chat, celui-ci n'est pas directement stocké, il est envoyé au serveur sous forme d'un paquet réseau. Le serveur reçoit le message, le stocke dans sa base de données permanente afin d'assurer un historique complet du chat. Ensuite, le serveur redistribue le message à l'ensemble des utilisateurs connectés au projet. C'est le handler réseau côté client qui se charge de l'ajouter au vecteur via `Chat::addMessage()`. L'affichage est alors entièrement reconstruit via `refreshChat()`.

L'envoi des messages est fait de manière asynchrone, afin d'éviter de bloquer le bon fonctionnement du système.

4 Edition de la carte

4.1 La navigation et la carte

La navigation et le zoom sont des actions locales, elles sont uniques pour chaque utilisateur et n'ont pas besoin d'être stockées dans le serveur. Chaque client va posséder sa propre vue (`sf::View`) qui représente la portion de l'écran visible pour l'utilisateur. Le déplacement modifie la position de la vue, donc son centre. Tandis que le zoom va modifier un facteur d'échelle sur celle-ci.

Le déplacement est géré par la classe `Move` qui stocke la position en x et y de l'utilisateur et va la mettre à jour selon les touches pressées. Il existe plusieurs moyens de se déplacer pour le client, il y a soit les touches ZQSD, soit les touches directionnelles, soit la minimap. Le déplacement a des limites afin d'éviter que l'utilisateur sorte de la carte.

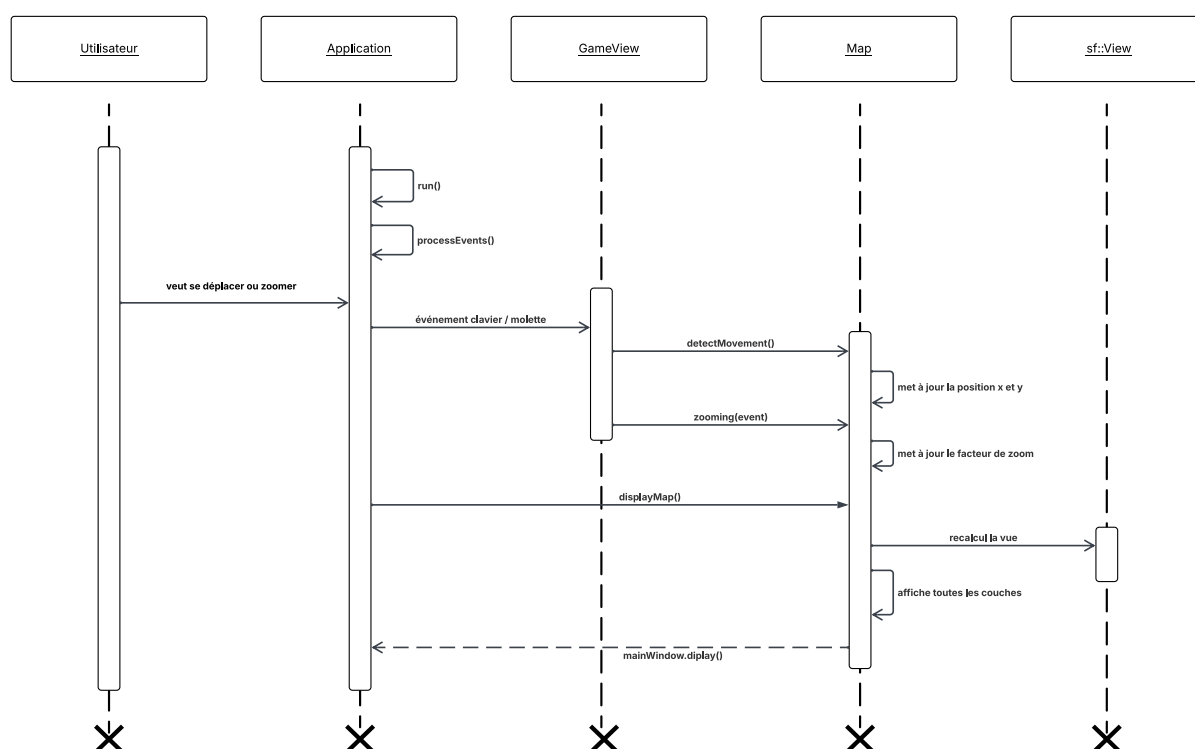


Fig. 4.1. – Diagramme de séquence du déplacement et du zoom

L'affichage de la carte est centralisé dans la classe `Map`. À chaque frame, celle-ci va recalculer la vue en tenant compte de sa position et de son zoom. De plus, celle-ci va afficher chaque couche sur la carte en vérifiant si celle-ci son masquée ou pas.

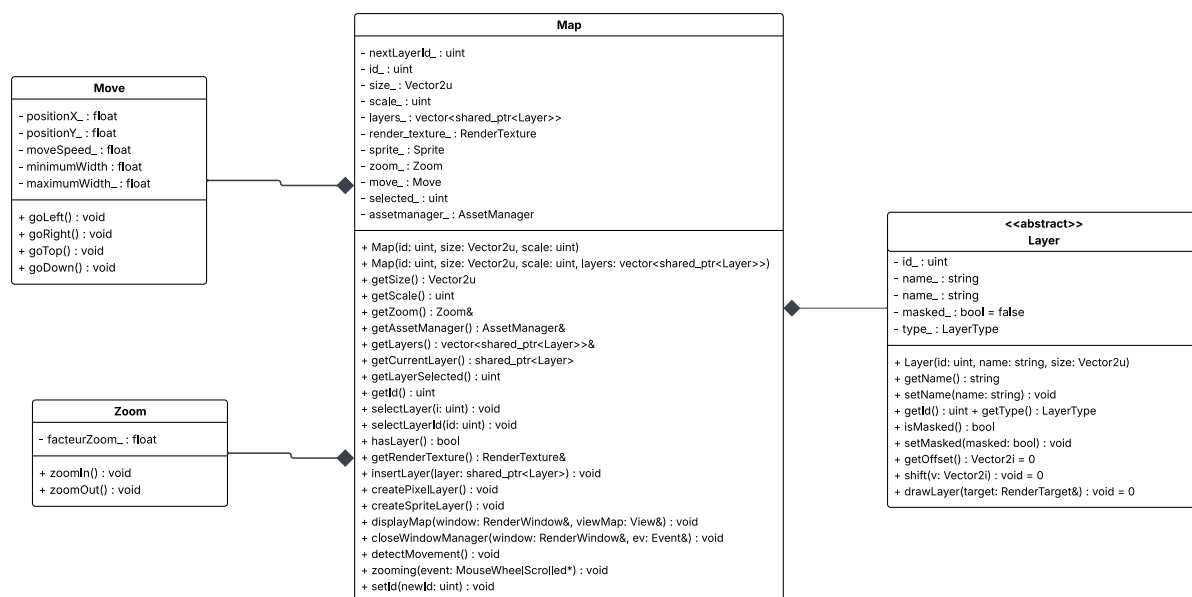


Fig. 4.2. – Diagramme de class de l'affichage des couches, déplacement et zoom

4.2 Les outils et les couches

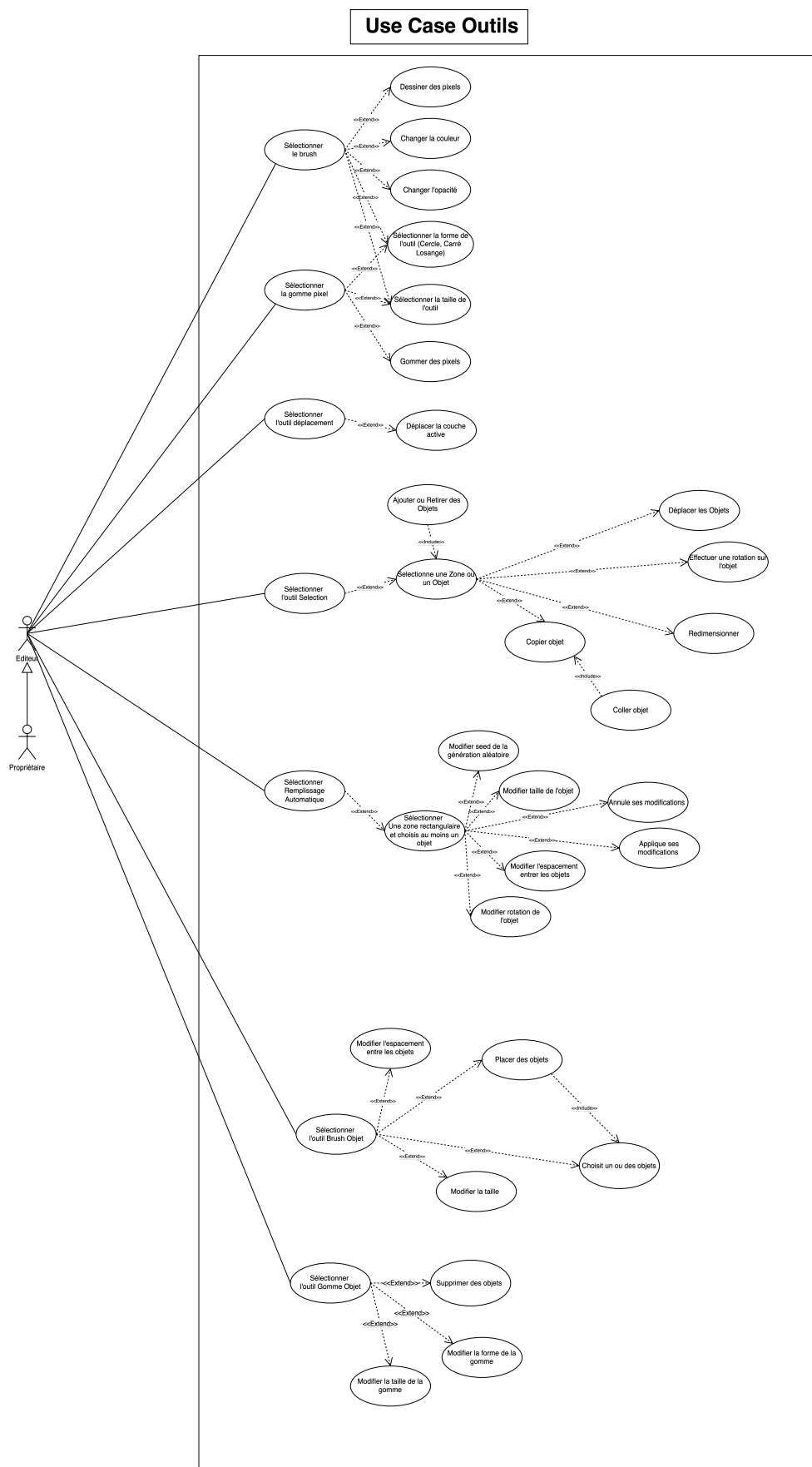


Fig. 4.3. – Use Case des outils


```

classDiagram
    class Project {
        +name : std::string
        +id : uint
        +window : sf::RenderWindow
        +gui : ImGui::Gui
        +viewMap : sf::View
        +viewId : uint
        +map : std::shared_ptr<Map>
        +toolbar : Toolbar
        +ProjectScale : unsigned int, size : sf::Vector2f, name : std::string, id : uint, window : sf::RenderWindow, gui : ImGui::Gui, manager : CInterfaceManager
        +getGui() : ImGui
        +getMap() : std::shared_ptr<Map>
        +getToolbar() : Toolbar
        +getView() : sf::View
        +setMapName : std::string
        +display()
        +displayScale()
        +displayToolbar()
        +displayAntiAli()
        +displayBackGround()
    }
    class Map {
        +headLayerId : uint = 0
        +id : uint
        +size : sf::Vector2f
        +scale : unsigned int
        +layers : std::vector<Layer>
        +tempLayer : std::shared_ptr<SpriteLayer>
        +render_texture : sf::RenderTarget
        +sprite : sf::Sprite
        +zoom : float
        +move : sf::Move
        +selected : unsigned int
        +zoomManager : ZoomManager
        +MapId : uint, size : sf::Vector2f, scale : unsigned int
        +MapId : uint, size : sf::Vector2f, scale : unsigned int, layers : std::vector<std::shared_ptr<Layer>>
        +getSprite() : sf::Sprite
        +getScale() : unsigned int
        +getZoom() : float
        +getZoomManager : ZoomManager
        +getLayer() : std::vector<std::shared_ptr<Layer>>
        +getTempLayer() : std::shared_ptr<SpriteLayer>
        +getLayerSelected() : unsigned int
        +getLayer(layer : std::uint) : std::shared_ptr<Layer>
        +getId() : uint
        +getTempLayer(layer : std::uint) : std::shared_ptr<SpriteLayer>
        +selectLayer() : unsigned int
        +selectLayerById : uint
        +headLayer : bool
        +const getRenderTexture() : const sf::RenderTarget
        +createMapLayer() : std::shared_ptr<Layer>
        +createMapLayer()
        +createSpriteLayer(layer_id : std::string name)
        +layer(layerId : uint)
        +layer(layerId : uint)
        +deleteLayer()
        +deleteLayer(layer_id : uint)
        +createSpriteLayer(layer_id : uint)
        +deleteLayer()
        +deleteLayer(layer_id : uint)
        +ZoomingEvent : sf::Event, MouseWheelScrolled const
        +closeRenderWindow(window : sf::RenderWindow, ev : sf::Event)
        +displayRenderWindow : sf::RenderWindow, viewMap : sf::View
    }
    class Layer {
        +name : std::string
        +id : uint
        +size : sf::Vector2f
        +masked : bool
        +fmap : bool
        +type : LayerType
        +Layer() : uint, name : std::string, size : sf::Vector2f
        +getMap() : std::string
        +setMapName : const std::string
        +setTemp()
        +getId() : const uint
        +getTemp() : LayerType
        +isMasked() : const bool
        +isTemp() : const bool
        +setMasked(masked : bool)
        +virtual getSprite() : const sf::Sprite = 0
        +virtual entity : sf::Sprite = 0
        +virtual drawLayerTarget : sf::RenderTarget = 0
        +virtual <Layer> : default
    }
    class PixelLayer {
        +texture : sf::Texture
        +other : sf::Sprite
        +PixelLayer(name : std::string, size : sf::Vector2f)
        +getTexture() : const sf::Texture
        +draw() : sf::Drawable
        +erase() : sf::Drawable
        +entity : sf::Sprite
        +drawLayerTarget : sf::RenderTarget
        +display()
    }
    class SpriteLayer {
        +sprite : sf::Sprite, mapPoint, SpriteObject, other : sf::Sprite
        +entity : uint
        +SpriteLayer(name : std::string, size : sf::Vector2f)
        +getSprite() : std::vector<SpriteObject>
        +getSprite() : sf::Sprite
        +draw() : const sf::Sprite, asset = "" : std::string
        +drawLayerTarget : sf::RenderTarget
        +erase() : sf::Drawable
        +entity : sf::Sprite
        +drawLayerTarget : sf::RenderTarget
        +addSprite : const sf::Sprite, id : uint
        +newSprite : uint, pos : sf::Vector2f, angle : float
        +setSprite() : std::vector<SpriteObject>
    }
    Project --> Map
    Map --> Layer
    Layer <|-- PixelLayer
    Layer <|-- SpriteLayer
  
```

The diagram illustrates the architecture of a 2D engine, showing the relationships between various components:

- Project** (Base Class):
 - Attributes: `name` (std::string), `id` (uint), `window` (sf::RenderWindow), `gui` (ImGui::Gui), `viewMap` (sf::View), `viewId` (uint), `map` (std::shared_ptr<Map>), `toolbar` (Toolbar).
 - Operations: `ProjectScale` (unsigned int, size: sf::Vector2f, name: std::string, id: uint, window: sf::RenderWindow, gui: ImGui::Gui, manager: CInterfaceManager), `getGui()` (ImGui), `getMap()` (std::shared_ptr<Map>), `getToolbar()` (Toolbar), `getView()` (sf::View), `setMapName` (std::string), `display()`, `displayScale()`, `displayToolbar()`, `displayAntiAli()`, `displayBackGround()`.
- Map** (Base Class):
 - Attributes: `headLayerId` (uint = 0), `id` (uint), `size` (sf::Vector2f), `scale` (unsigned int), `layers` (std::vector<Layer>), `tempLayer` (std::shared_ptr<SpriteLayer>), `render_texture` (sf::RenderTarget), `sprite` (sf::Sprite), `zoom` (float), `move` (sf::Move), `selected` (unsigned int), `zoomManager` (ZoomManager).
 - Operations: `MapId` (uint, size: sf::Vector2f, scale: unsigned int), `MapId` (uint, size: sf::Vector2f, scale: unsigned int, layers: std::vector<std::shared_ptr<Layer>>), `getSprite()` (sf::Sprite), `getScale()` (unsigned int), `getZoom()` (float), `getZoomManager` (ZoomManager), `getLayer()` (std::vector<std::shared_ptr<Layer>>), `getTempLayer()` (std::shared_ptr<SpriteLayer>), `getLayerSelected()` (unsigned int), `getLayer(layer: std::uint)` (std::shared_ptr<Layer>), `getId()` (uint), `getTempLayer(layer: std::uint)` (std::shared_ptr<SpriteLayer>), `selectLayer()` (unsigned int), `selectLayerById` (uint), `headLayer` (bool), `const getRenderTexture()` (const sf::RenderTarget), `createMapLayer()` (std::shared_ptr<Layer>), `createMapLayer()`, `createSpriteLayer(layer_id: std::string name)`, `layer(layerId: uint)`, `layer(layerId: uint)`, `deleteLayer()`, `deleteLayer(layer_id: uint)`, `createSpriteLayer(layer_id: uint)`, `deleteLayer()`, `deleteLayer(layer_id: uint)`, `ZoomingEvent` (sf::Event, MouseWheelScrolled const), `closeRenderWindow(window: sf::RenderWindow, ev: sf::Event)`, `displayRenderWindow` (sf::RenderWindow, viewMap: sf::View).
- Layer** (Base Class):
 - Attributes: `name` (std::string), `id` (uint), `size` (sf::Vector2f), `masked` (bool), `fmap` (bool), `type` (LayerType).
 - Operations: `Layer()` (uint, name: std::string, size: sf::Vector2f), `getMap()` (std::string), `setMapName` (const std::string), `setTemp()`, `getId()` (const uint), `getTemp()` (LayerType), `isMasked()` (const bool), `isTemp()` (const bool), `setMasked(masked: bool)`, `virtual getSprite()` (const sf::Sprite = 0), `virtual entity` (sf::Sprite = 0), `virtual drawLayerTarget` (sf::RenderTarget = 0), `virtual <Layer>` (default).
- PixelLayer** (Derived Class):
 - Attributes: `texture` (sf::Texture), `other` (sf::Sprite).
 - Operations: `PixelLayer(name: std::string, size: sf::Vector2f)`, `getTexture()` (const sf::Texture), `draw()` (sf::Drawable), `erase()` (sf::Drawable), `entity` (sf::Sprite), `drawLayerTarget` (sf::RenderTarget), `display()`.
- SpriteLayer** (Derived Class):
 - Attributes: `sprite` (sf::Sprite, mapPoint, SpriteObject, other: sf::Sprite), `entity` (uint).
 - Operations: `SpriteLayer(name: std::string, size: sf::Vector2f)`, `getSprite()` (std::vector<SpriteObject>), `getSprite()` (sf::Sprite), `draw()` (const sf::Sprite, asset = "" : std::string), `drawLayerTarget` (sf::RenderTarget), `erase()` (sf::Drawable), `entity` (sf::Sprite), `drawLayerTarget` (sf::RenderTarget), `addSprite` (const sf::Sprite, id: uint), `newSprite` (uint, pos: sf::Vector2f, angle: float), `setSprite()` (std::vector<SpriteObject>).

Fig. 4.4. – Diagramme de classe des outils et couches

Utilisation Outils

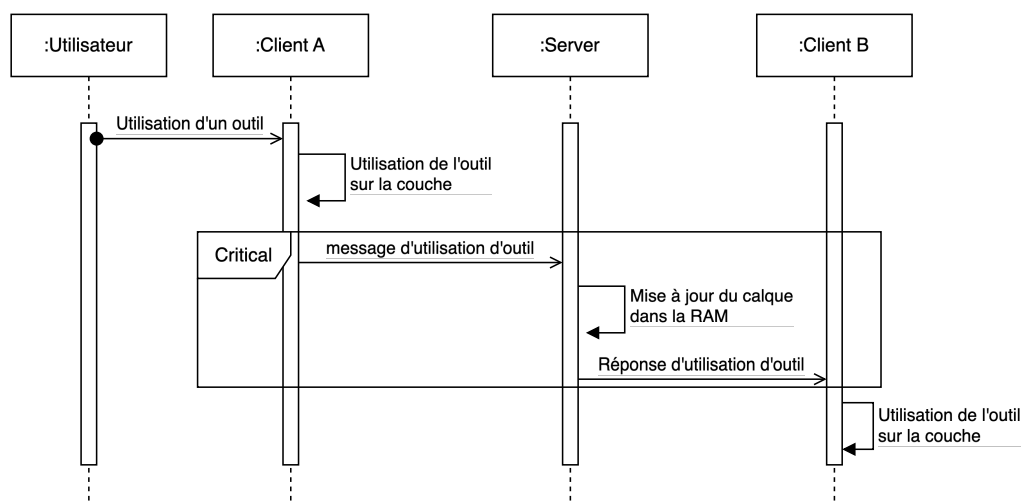


Fig. 4.5. – Diagramme de séquence de l'utilisation des outils

4.3 Le remplissage automatique

Remplissage Automatique

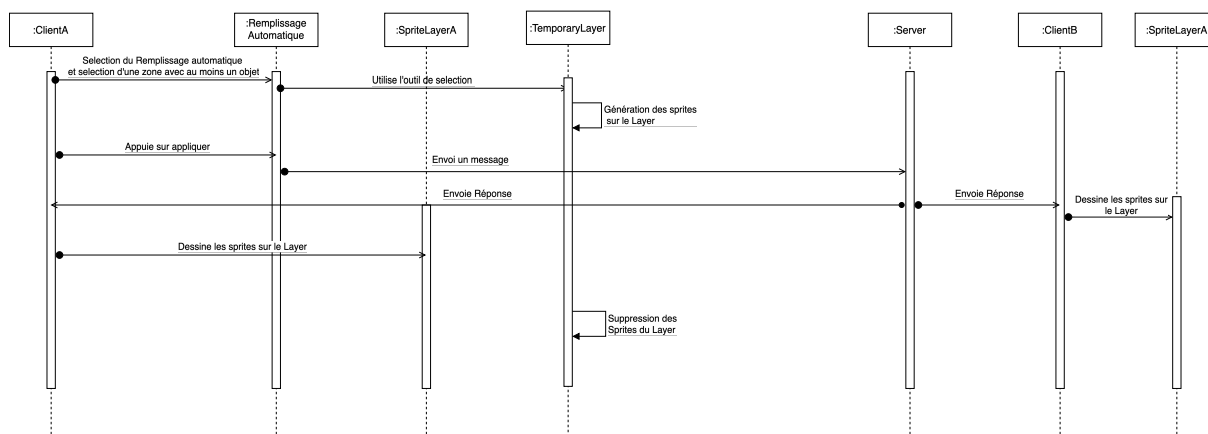


Fig. 4.6. – Sequence Diagram Automatic Filling

5 Spécification techniques

5.1 Stockage des données

Le stockage des données se fera directement par fichier et en faisant appel à un Système de Gestion de Base de Données (SGBD).

En effet, la partie “image” projet peut être décomposé en un dossier contenant les médias image qui ont été importés dans le projet, un dossier sprite, contenant un encodage du contenu des différents calques pixels et un fichier JSON qui donne la structure des différents calques (position, agrandissement, niveau, nom dans les fichiers de l’image à charger, etc). La partie “communautaire” (c’est-à-dire le chat, les participants et leurs rôles) est à part, encodée dans 2 fichier: l’un contenant la liste des participants avec leur rôles et un autre contenant l’entièreté du chat.

Ce découpage rend le maintient des données et leur modifications conceptuellement assez simple: importer une image ajoute un fichier dans le dossier image, de même pour l’ajout de pixel tandis que la modification de paramètre de calque modifie un objet mémoire qui contient la liste des calques du projet. Les sauvegardes ponctuelles ne correspondent qu’à une conversion de l’objet “projet” en RAM en un fichier JSON, les images et le contenu des calques pixels étant à priori sauver sur le disque dur lors de leur envoi au serveur.

La gestion des ID des utilisateurs, projets et de leurs rôles/permissions se fera à l’aide d’une base de donnée SQL. Qt nous offre les outils pour la manipuler efficacement, ce qui simplifie grandement son utilisation tout en améliorant notre implémentation du serveur.

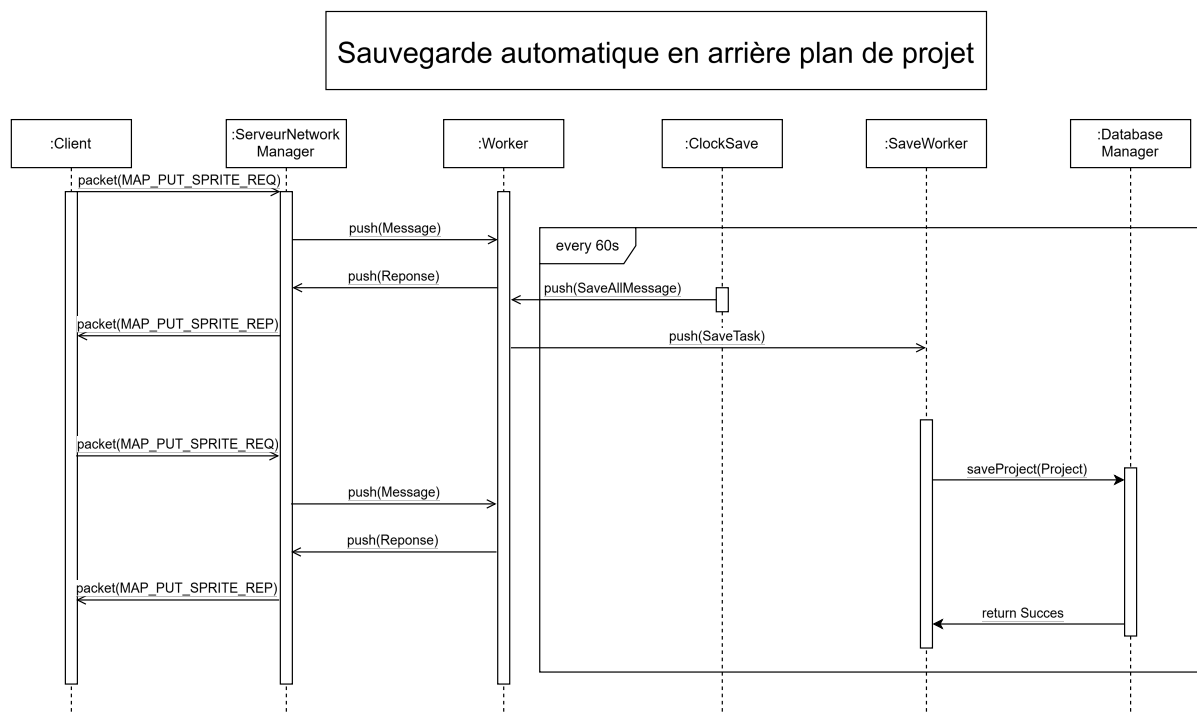


Fig. 5.1. – Diagramme de séquence de la sauvegarde automatique

Pour les sauvegardes régulières, le serveur fait appel à un thread qui ne s’occupe exclusivement que des demandes liée à la lecture ou le chargement de données depuis le disque dur.

Le faire tourner dans un thread séparé nous assure que celui-ci est asynchrone au demande de modification. Ainsi nous sommes sûr qu'une demande de sauvegarde ne figera pas la synchronisation des modification à cause du "long" temps d'écriture.

5.2 Gestion de la concurrence des modifications

Il nous suffit de gérer les requêtes de modification dans l'ordre de leur arrivée et des les placer dans un structure de file (premier rentré, premier sortie).

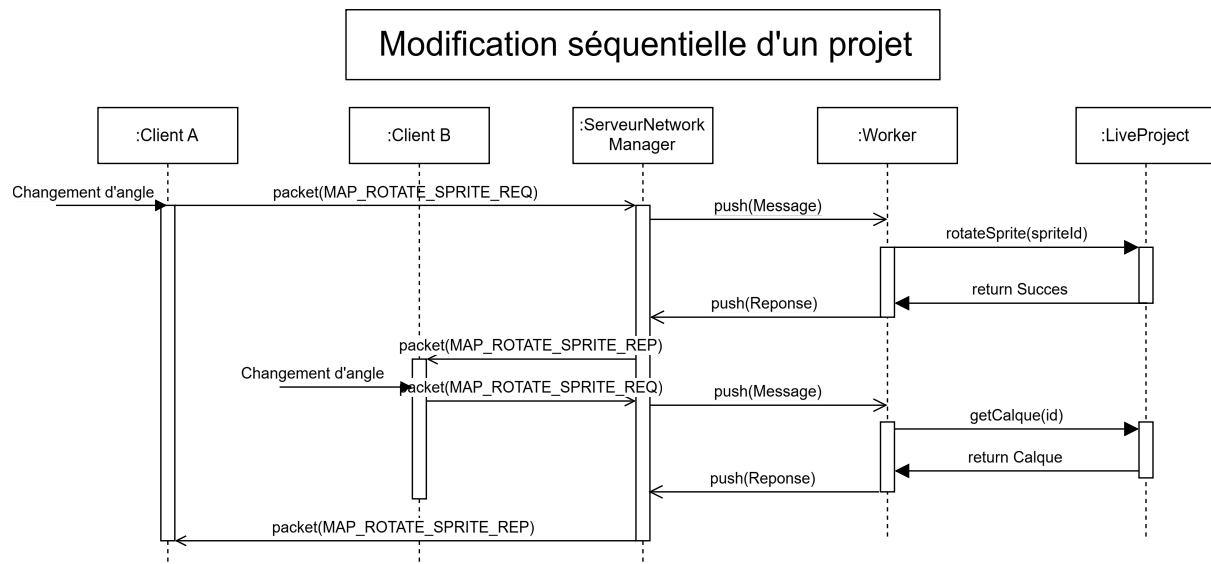


Fig. 5.2. – Diagramme de séquence sur les modification d'un projet

5.3 Gestion du flux de données

Afin de mettre en place tout ces threads au sein du serveur, plusieurs classes mère de « communication » on été codé. Chacune d'entre elle définissent une méthode « d'auto-gestion ». Ainsi ajouter un nouveau message ne nécessite pas de modifier le code de « l'exécuteur », il suffit de correctement implémenter cette méthode qui est appelé quand le message doit être résolu. Ces classes structures la communications entre tous les threads du serveur. Cette approche à aussi permis de très simplement codé la sauvegarde automatique. Elle demande un thread qui envoie simplement un message toutes les 30 secondes.

Schéma du flux de donnée du serveur

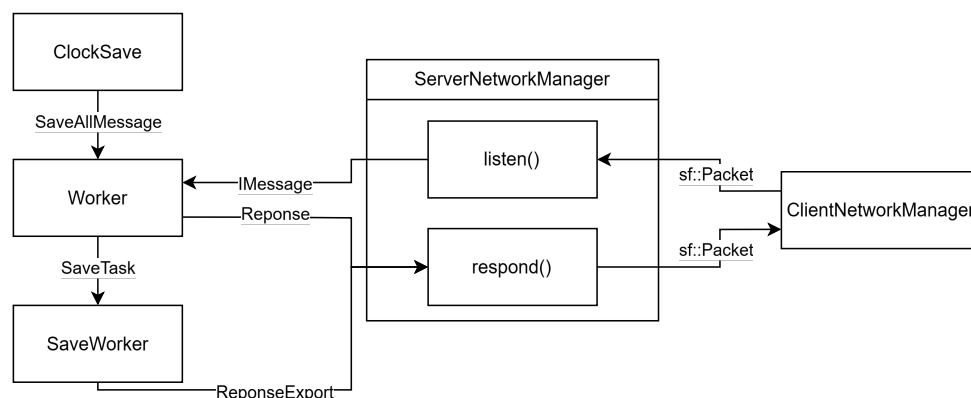


Fig. 5.3. – Diagramme de séquence sur les modification d'un projet

6 Annexes

Pour ce projet, nous utilisons la bibliothèque SFML. Elle nous permettra d'avoir un rendu graphique, capturer les interactions de l'utilisateur par sa souris ou clavier, ainsi que l'envoi des données vers le serveur et inversement, etc.

Bibliographie

- [1] J. L. Taylor, *Complex Variables*. AMS, 2011.
- [2] E. Chen, « LaTeX FAQs ». [En ligne]. Disponible sur: <https://web.evanchen.cc/faq-latex.html>
- [3] A. Gleave, « Writing Beautifully in LaTeX ». [En ligne]. Disponible sur: <https://www.gleave.me/post/latex-design-patterns/>
- [4] OpenAI, « ChatGPT ». [En ligne]. Disponible sur: <https://chatgpt.com/>
- [5] Antropic, « Claude ». [En ligne]. Disponible sur: <https://claude.ai/new>
- [6] SFML-dev, « Tutorials for SFML 3.1 ». [En ligne]. Disponible sur: <https://www.sfm1-dev.org/tutorials/3.1/>
- [7] TGUI, « Class List ». [En ligne]. Disponible sur: <https://tgui.eu/documentation/latest-stable/annotated.html>